GPUMODE Lecture Study Notes: Lecture 34 Low Bit Triton Kernels

Core Problem the Lecture Addresses

This lecture explains how to write fast **low-bit Triton kernels** for quantized matrix multiplication, especially for transformer inference. The main practical problem is that modern large language models and vision-language models are dominated by linear layers. These layers contain most of the parameters and account for much of the runtime cost. If the weights remain in FP16 or BF16, serving large models requires expensive GPU memory capacity and high memory bandwidth.

The central engineering question is:

> How can we run quantized transformer linear layers efficiently in Triton while supporting packed weights, low-bit formats, grouping, zero points, scales, multiple batch-size regimes, and PyTorch production integration?

The speaker focuses on an open-source project called `gemlite`. The goal is not only to quantize the model weights, but to run the quantized model quickly. A naive implementation might first dequantize all weights into FP16 and then call a normal matrix multiplication routine. That approach defeats the purpose of quantization because it materializes the full-precision weight matrix again.

The desired fused computation is

$$C = X\widehat{W}, \qquad \widehat{W} = S \odot (W_q - Z),$$

where X is the activation matrix, W_q is the quantized weight matrix, Z is the zero-point tensor, S is the scale tensor, and $\widehat{W}$ is the dequantized approximation of the original full-precision weight matrix.

A high-performance low-bit kernel should load the compact packed representation, unpack it inside the kernel, dequantize it inside the kernel, immediately multiply it by activations, and avoid writing the full dequantized matrix back to memory.

Required Background

Transformer Linear Layers

A transformer model contains many linear layers in attention projections and feed-forward blocks. A linear layer has the form

$$Y = XW + b,$$

where

$$X \in \mathbb{R}^{M \times K}, \qquad W \in \mathbb{R}^{K \times N}, \qquad Y \in \mathbb{R}^{M \times N}.$$

For kernel design, the bias term is usually ignored or fused separately, so the core operation is

$$Y = XW.$$

The model weights W are large and reused throughout inference. Reducing their storage size reduces VRAM usage and can improve runtime when weight loading is the bottleneck.

Prefill and Decode

Transformer inference has two major phases.

Prefill processes the input prompt. If the prompt has many tokens, then M is relatively large:

$$X \in \mathbb{R}^{M \times K}, \qquad M \gg 1.$$

The corresponding linear operation is a matrix-matrix multiplication. It often has enough arithmetic work to be compute-bound or at least to make tensor core utilization important.

Decode generates one or a few tokens at a time. In the simplest case,

$$x \in \mathbb{R}^{1 \times K}, \qquad W \in \mathbb{R}^{K \times N}, \qquad y \in \mathbb{R}^{1 \times N}.$$

This is closer to matrix-vector multiplication. It is usually memory-bound because every generated token requires reading large weight matrices, while the amount of arithmetic per loaded byte is small.

Memory-Bound Versus Compute-Bound Execution

The roofline model gives the basic performance intuition:

$$\text{Achievable FLOP/s} = \min\left(\text{Peak Compute},\ \text{Memory Bandwidth} \times \text{Arithmetic Intensity}\right).$$

The arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

Low-bit quantization reduces the number of bytes moved for weights. This is especially valuable in decode, where arithmetic intensity is low and global memory bandwidth is often the limiting factor.

Triton Programming Model

Triton exposes a block-programming model. Rather than writing code for individual CUDA threads, the programmer writes a program instance that operates on a tile or block of data. A typical Triton kernel does the following:

1. Computes a program identifier using `tl.program_id`.
2. Maps that program identifier to a tile of the output matrix.
3. Computes pointer offsets for input and output tiles.
4. Loads tiles using `tl.load`.
5. Performs vectorized arithmetic or block matrix multiplication using `tl.dot`.
6. Stores results using `tl.store` or accumulates using `tl.atomic_add`.

Triton makes experimentation easier than CUDA, but the lecture emphasizes that Triton does not automatically produce optimal kernels. Low-level performance details still matter.

Main Concepts

Low-Bit Quantization

Quantization reduces the number of bits used to represent values. Instead of storing weights in FP16 or FP32, one may store them in INT8, FP8, INT4, INT3, INT2, or INT1-like representations.

For linear quantization, the dequantized value is computed using

$$\widehat{w} = s(w_q - z),$$

where w_q is the quantized value, s is the scale, and z is the zero point.

For a matrix, this becomes

$$\widehat{W} = S \odot (W_q - Z).$$

The quantized matrix W_q is compact, but the kernel must still reconstruct an approximate full-precision value before using it in arithmetic.

Symmetric and Asymmetric Quantization

In asymmetric quantization, both scales and zero points are used:

$$\widehat{W}_{k,n} = S_{k,n}(W_{q,k,n} - Z_{k,n}).$$

In symmetric quantization, the zero point is absent or effectively zero:

$$\widehat{W}_{k,n} = S_{k,n}W_{q,k,n}.$$

Symmetric quantization is often faster because the kernel performs fewer operations and may load fewer tensors. However, for very low bit widths such as 4-bit, 3-bit, or 2-bit, zero points can be important for accuracy.

Grouped Quantization

Channel-wise quantization assigns one scale and zero point per output channel. This is simple but can lose accuracy at very low bit widths. Grouped quantization assigns separate scales and zero points to smaller groups along the K-dimension.

Let g be the group size. Then the group index for element k is

$$r = \left\lfloor \frac{k}{g} \right\rfloor.$$

The dequantization equation is

$$\widehat{W}_{k,n} = S_{r,n}\left(W_{q,k,n} - Z_{r,n}\right), \qquad r = \left\lfloor \frac{k}{g} \right\rfloor.$$

When $g = K$, the method reduces to channel-wise quantization. When g is smaller, quantization becomes more locally adaptive and usually more accurate, but the kernel must load scale and zero-point tensors more frequently.

Bit Packing

Low-bit values usually cannot be stored directly as native PyTorch tensor dtypes. For example, PyTorch does not generally store an ordinary tensor as native INT4 values. Therefore, low-bit values are packed into larger integer containers.

For two 4-bit values q_0 and q_1, one can pack them into one 8-bit value:

$$p = q_0 \mid (q_1 \ll 4).$$

The corresponding unpacking operations are

$$q_0 = p \,\&\, 15, \qquad q_1 = (p \gg 4) \,\&\, 15.$$

If 4-bit values are packed into a 32-bit integer, then each 32-bit container holds

$$\frac{32}{4} = 8$$

values. More generally, for a bit width b, the number of values per 32-bit container is

$$E = \frac{32}{b},$$

assuming b divides 32.

```
def pack_two_int4(q0, q1):
    # q0 and q1 are assumed to be in the range [0, 15].
    low = q0 & 0xF
    high = (q1 & 0xF) << 4
    return low | high

def unpack_two_int4(packed):
    q0 = packed & 0xF
    q1 = (packed >> 4) & 0xF
    return q0, q1
```

Packing is lossless with respect to the quantized integers. The approximation error comes from quantization, not from packing.

Why Packing Format Matters

The packing format affects kernel performance. A packing layout that is ideal for a CUDA matrix-vector kernel may not be ideal for a Triton matrix-matrix kernel. If prefill and decode kernels require different packing formats, the system may

need to store multiple packed copies of the weights or repack between phases. Both choices are unattractive.

The lecture stresses the importance of one shared packing format that can support:

- GEMM for prefill.
- GMV for single-token decode.
- Split-K GEMM for batched decode.
- Different activation dtypes.
- Different weight bit widths.
- Different GPU architectures.

Repacking between prefill and decode is possible, but it can be slow. It also requires knowing exactly how both kernels expect weights to be packed.

Hardware-Level Explanation

GPU Memory Hierarchy

A GPU kernel interacts with several levels of memory and execution hardware:

- **Global memory**: off-chip memory such as HBM or GDDR.
- **L2 cache**: shared cache across streaming multiprocessors.
- **L1 cache and shared memory**: on-chip memory local to an SM.
- **Registers**: fastest storage, local to executing threads.
- **CUDA cores**: scalar and vector arithmetic units.
- **Tensor cores**: matrix multiply units for supported tile shapes and dtypes.

Low-bit kernels reduce global memory traffic by reading packed weights, but they introduce extra work inside the SM: unpacking, shifting, masking, casting, and dequantization.

Why Decode Benefits Strongly From Quantization

In decode, a single activation vector multiplies a large weight matrix:

$$y_n = \sum_{k=0}^{K-1} x_k W_{k,n}.$$

The kernel reads $K \times N$ weights for each generated token. The amount of computation is approximately $2KN$ floating-point operations, but the batch

dimension is too small to reuse weights across many activation rows. Therefore, reducing the bytes per weight can give a large speedup.

If FP16 weights require

$$2KN$$

bytes and b-bit weights require ideally

$$\frac{bKN}{8}$$

bytes, then the ideal memory reduction factor is

$$\frac{2KN}{bKN/8} = \frac{16}{b}.$$

Thus INT4 can ideally reduce weight memory traffic by $4\times$, while INT2 can ideally reduce it by $8\times$, ignoring scale and zero-point overhead.

Why Prefill Has Different Requirements

In prefill, M can be large:

$$Y_{m,n} = \sum_{k=0}^{K-1} X_{m,k} W_{k,n}.$$

The same weight tile can be reused across many rows of X. This increases arithmetic intensity and makes tensor core throughput more important. Therefore, a prefill kernel should usually look like a tiled GEMM kernel using `tl.dot`.

Tensor Cores and `tl.dot`

Triton expresses matrix multiplication through `tl.dot`. A GEMM tile computes

$$C_{\text{tile}} \leftarrow C_{\text{tile}} + A_{\text{tile}} B_{\text{tile}},$$

where

$$A_{\text{tile}} \in \mathbb{R}^{B_M \times B_K}, \qquad B_{\text{tile}} \in \mathbb{R}^{B_K \times B_N}, \qquad C_{\text{tile}} \in \mathbb{R}^{B_M \times B_N}.$$

For a decode-style GMV kernel, the operation is more vector-like. Padding it into a fake matrix multiplication can waste work. The lecture describes using elementwise multiplication followed by `tl.sum` for GMV instead of forcing everything into `tl.dot`.

Cache and Eviction Policy

Triton gives limited explicit cache control. One important mechanism is the `eviction_policy` argument to `tl.load`. Two useful policies are:

- `evict_last`: keep this data in cache longer if possible.
- `evict_first`: evict this data earlier if possible.

For decode and split-K, activations are small and reused. Therefore, loading activations with `evict_last` can improve performance. Weights are larger and more stream-like, so they are often less useful to keep cached.

Visual Concept

Draw packed low-bit weights, scale tensors, zero-point tensors, and activation tensors in global memory. Show each Triton program loading a tile into an SM. Inside the SM, draw unpacking, dequantization, dot product or vector reduction, accumulation, and finally either a store or atomic add to the output.

Mathematical Reasoning

Fused Quantized Matrix Multiplication

The original dense operation is

$$Y = XW.$$

With quantized weights, the kernel computes

$$Y = X\widehat{W}, \qquad \widehat{W} = S \odot (W_q - Z).$$

At the element level,

$$Y_{m,n} = \sum_{k=0}^{K-1} X_{m,k} S_{\lfloor k/g \rfloor, n} \left(W_{q,k,n} - Z_{\lfloor k/g \rfloor, n} \right).$$

A fused kernel computes this expression without storing $\widehat{W}$ in global memory.

Channel-Wise Symmetric Case

If the quantization is channel-wise and symmetric, then

$$\widehat{W}_{k,n} = S_n W_{q,k,n}.$$

The output is

$$Y_{m,n} = S_n \sum_{k=0}^{K-1} X_{m,k} W_{q,k,n}.$$

This is cheaper because S_n can be applied after the inner product rather than loaded and applied for every group inside the loop.

Grouped Case

For grouped quantization,

$$Y_{m,n} = \sum_r S_{r,n} \sum_{k \in \mathcal{G}_r} X_{m,k} \left(W_{q,k,n} - Z_{r,n} \right).$$

The scale and zero point depend on the group r. This creates more memory loads and more arithmetic inside the kernel. It is usually more accurate than channel-wise low-bit quantization, but it can be slower.

Activation Quantization

If activations are also quantized, one may have

$$\widehat{X}_{m,k} = S_m^X \left(X_{q,m,k} - Z_m^X \right).$$

For symmetric activation and weight quantization,

$$Y_{m,n} = S_m^X S_n^W \sum_{k=0}^{K-1} X_{q,m,k} W_{q,k,n}.$$

The product $S_m^X S_n^W$ is an outer product over rows and columns. This outer product is usually cheap compared with the full matrix multiplication.

Split-K Reduction

Split-K divides the reduction dimension into multiple pieces. The normal GEMM output is

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

With P splits,

$$C_{m,n} = \sum_{p=0}^{P-1} C_{m,n}^{(p)},$$

where

$$C_{m,n}^{(p)} = \sum_{k \in \mathcal{K}_p} A_{m,k} B_{k,n}.$$

Each program computes a partial result and accumulates it using atomic addition:

$$C_{m,n} \leftarrow C_{m,n} + C_{m,n}^{(p)}.$$

Because atomic addition adds to the existing value, the output must first be initialized to zero.

Code Walkthrough

Naive Dequantize-Then-Matmul

The simplest implementation is correct but inefficient:

```
def slow_quantized_linear(x, wq, scales, zeros):
    w_hat = (wq.float() - zeros.float()) * scales.float()
    return x @ w_hat
```

This is slow because the dequantized matrix `w_hat` is materialized. That loses the memory-bandwidth advantage of low-bit storage.

Fused Kernel Pseudocode

A fused low-bit kernel performs unpacking and dequantization inside the matrix multiplication loop:

```
acc = zeros((BLOCK_M, BLOCK_N), dtype=float32)

for k0 in range(0, K, BLOCK_K):
    a = load_activation_tile(A, k0)

    packed_b = load_packed_weight_tile(B_packed, k0)
    bq = unpack_lowbit(packed_b)

    s = load_scale_tile(scales, k0)
    z = load_zero_tile(zeros, k0)

    b = (bq - z) * s
```

```
    acc += dot(a, b)

store(C, acc)
```

The key point is that b is dequantized only inside the program instance and only for the current tile. The full dequantized matrix is never written to global memory.

Simplified Triton GEMM Structure

The following is an educational sketch of a standard Triton matrix multiplication structure. It is not a complete production kernel, but it shows the mapping from program IDs to output tiles.

```
@triton.jit
def matmul_kernel(A, B, C,
                  M: tl.constexpr, N: tl.constexpr,
                  K: tl.constexpr,
                  BLOCK_M: tl.constexpr,
                  BLOCK_N: tl.constexpr,
                  BLOCK_K: tl.constexpr):
    pid = tl.program_id(0)

    num_pid_n = tl.cdiv(N, BLOCK_N)
    pid_m = pid // num_pid_n
    pid_n = pid % num_pid_n

    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)

    acc = tl.zeros((BLOCK_M, BLOCK_N), tl.float32)

    for k0 in tl.range(0, K, BLOCK_K):
        a_ptrs = A + offs_m[:, None] * K + k0 + offs_k[None, :]
        b_ptrs = B + (k0 + offs_k[:, None]) * N + offs_n[None, :]

        a = tl.load(a_ptrs, mask=offs_m[:, None] < M, other=0.0)
        b = tl.load(b_ptrs, mask=offs_n[None, :] < N, other=0.0)

        acc += tl.dot(a, b)

    c_ptrs = C + offs_m[:, None] * N + offs_n[None, :]
    mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    tl.store(c_ptrs, acc, mask=mask)
```

For low-bit weights, the `tl.load` of B is replaced by a packed load, bit extraction,

casting, scale loading, zero-point loading, and dequantization.

Simplified Low-Bit Unpacking Logic

For 4-bit values packed into 32-bit containers, each packed value contains eight quantized weights. The unpacking logic uses shifts and masks:

```
# packed contains int32 containers.
# shifts is [0, 4, 8, 12, 16, 20, 24, 28].
q = (packed[:, :, None] >> shifts[None, None, :]) & 0xF
q = tl.reshape(q, (BLOCK_K, BLOCK_N))

w = (q.to(tl.float32) - z) * s
```

A practical issue is that the packed tile shape is smaller than the logical unpacked tile shape. The lecture describes using repeated or interleaved indices so that Triton loads packed containers in a pattern that expands naturally into the target logical tile.

For example, for eight 4-bit values per 32-bit word, the offsets may repeat like

$$0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2.$$

This can avoid expensive concatenation steps, but it makes block pointers and TMA harder to use.

GMV Kernel for Decode

For batch size one, the operation is

$$y_n = \sum_{k=0}^{K-1} x_k \widehat{W}_{k,n}.$$

A GMV-style kernel can launch many programs over K-chunks and N-chunks. Each program computes a partial dot product and atomically adds into the output.

```
@triton.jit
def gmv_like_kernel(X, Wq, S, Z, Y,
                    K: tl.constexpr, N: tl.constexpr,
                    BLOCK_K: tl.constexpr,
                    BLOCK_N: tl.constexpr):
    pid_k = tl.program_id(0)
    pid_n = tl.program_id(1)

    offs_k = pid_k * BLOCK_K + tl.arange(0, BLOCK_K)
    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
```

```
    x = tl.load(
        X + offs_k,
        mask=offs_k < K,
        other=0.0,
        eviction_policy="evict_last"
    )

    wq = load_and_unpack_weight_block(Wq, offs_k, offs_n)
    s = load_scale_block(S, offs_k, offs_n)
    z = load_zero_block(Z, offs_k, offs_n)

    w = (wq - z) * s
    partial = tl.sum(x[:, None] * w, axis=0)

    tl.atomic_add(Y + offs_n, partial, mask=offs_n < N)
```

This kernel does not need tensor cores. It is memory-bound, so the priority is loading packed weights and reused activations efficiently.

Split-K GEMM for Small Batches

For small batch sizes, split-K GEMM is a hybrid between GMV and GEMM. It uses `tl.dot`, but it parallelizes over the reduction dimension and combines partial results with atomics.

```
@triton.jit
def splitk_gemm_kernel(A, Wq, S, Z, C,
                       M: tl.constexpr, N: tl.constexpr,
                       K: tl.constexpr,
                       SPLIT_K: tl.constexpr,
                       BLOCK_M: tl.constexpr,
                       BLOCK_N: tl.constexpr,
                       BLOCK_K: tl.constexpr):
    pid_mn = tl.program_id(0)
    pid_split = tl.program_id(1)

    # Map pid_mn to an output tile.
    # Map pid_split to a slice of the K dimension.
    # Load A tile, packed W tile, scales, and zeros.
    # Unpack and dequantize W.
    # Compute a partial matrix product with tl.dot.
    # Accumulate into C with tl.atomic_add.
```

The important correctness requirement is that the output tensor must be zero before launching the kernel:

$$C \leftarrow 0.$$

Performance Intuition

Triton Is Productive, But Not Magic

The lecture repeatedly emphasizes that Triton makes kernels easier to write and debug, but high performance still requires GPU systems knowledge. A correct low-bit Triton kernel may perform poorly unless the programmer controls memory layout, packing format, block sizes, caching hints, load order, and autotuning.

Memory Layout Can Dominate Performance

If the kernel expects column-major-like weights, simply transposing a PyTorch tensor is not enough. A transpose often creates a non-contiguous view. The correct preparation is

```
B_col_major = B.T.contiguous()
```

The `contiguous` call can be essential. A non-contiguous layout can cause inefficient memory access and severely reduce performance.

Load Order Can Affect Runtime

A surprising observation from the lecture is that changing the order of loads can affect runtime. For example:

```
if LOAD_ORDER == 0:
    a = tl.load(a_ptrs, eviction_policy="evict_last")
    b = tl.load(b_ptrs)
else:
    b = tl.load(b_ptrs)
    a = tl.load(a_ptrs, eviction_policy="evict_last")
```

The best order may depend on the GPU and kernel. Therefore, the speaker suggests treating load order as an autotuning parameter.

Eviction Policy Can Help Decode

For low batch sizes, activations are small and reused. Loading them with

```
eviction_policy="evict_last"
```

can improve performance by encouraging cache retention. This is similar in spirit to manually caching small activation vectors in shared memory in CUDA.

Grouped Quantization Has a Runtime Cost

Grouped quantization improves accuracy, but scales and zeros may need to be loaded inside the K-loop. If the group size is small, the kernel performs more scale and zero-point loads. Channel-wise or symmetric quantization can be faster because the scale may be applied after accumulation.

Packing Scales and Zeros Together May Not Help

One tempting optimization is to pack FP16 scales and FP16 zeros into one 32-bit value. This reduces the number of memory loads, but the lecture reports that in Triton this can be slower because unpacking and temporary tensor manipulation cost more than the saved load instruction.

Autotuning Must Be Pruned Carefully

Autotuning configurations must be compatible with the shape and quantization scheme. For example, a simple split-K kernel may require

$$K \equiv 0 \pmod{B_K \cdot P},$$

where B_K is the block size in the K-dimension and P is the split-K factor.

Grouped quantization also constrains valid block sizes. If the group size is g, then the tile shape must not cause incorrect scale or zero-point usage. Invalid autotuning configurations may produce wrong answers instead of crashing.

Atomic Add Requires Care

For GMV and split-K, many programs contribute to the same output. The output update is

$$C_{m,n} \leftarrow C_{m,n} + C^{(p)}_{m,n}.$$

This requires atomic addition and zero initialization. The speaker also notes that default settings for some Triton operations are not always fastest. However, atomic memory semantics should be changed carefully because they affect correctness guarantees.

Three-Bit and Five-Bit Kernels

Power-of-two bit widths such as 1, 2, 4, and 8 are easier to pack. Three-bit and five-bit formats are harder. One approach for 3-bit quantization is to split each value into a 2-bit part and a 1-bit part:

$$q = q_{\text{low2}} + 4q_{\text{high1}},$$

where

$$q_{\text{low2}} = q \bmod 4, \qquad q_{\text{high1}} = \left\lfloor \frac{q}{4} \right\rfloor .$$

Then the kernel loads two packed matrices and reconstructs the 3-bit value. This increases kernel complexity but may still be faster than 4-bit when decode is strongly memory-bound.

Production Integration

A Kernel Is Not the Whole System

A production-ready quantized inference system needs more than a fast Triton kernel. It must also handle

- kernel selection by shape and batch size,
- autotuning and autotune-result caching,
- PyTorch integration,
- `torch.compile` compatibility,
- CUDA graph compatibility,
- serialization of selected configurations,
- consistent packing formats,
- integration with model and serving frameworks.

`torch.compile` and Custom Ops

Some Triton features used for performance may not be supported cleanly by `torch.compile`. The lecture mentions pre-hooks and early config pruning as examples. A workaround is to wrap the Triton kernel in a PyTorch custom op and register a fake tensor implementation.

```
import torch

@torch.library.custom_op("gemlite::quant_matmul", mutates_args=())
def quant_matmul(x, packed_w, scales, zeros):
    return call_triton_kernel(x, packed_w, scales, zeros)

@quant_matmul.register_fake
def quant_matmul_fake(x, packed_w, scales, zeros):
    m = x.shape[0]
    n = infer_output_features(packed_w)
    return torch.empty((m, n), device=x.device, dtype=x.dtype)
```

The downside is that `torch.compile` may no longer see through the custom op. As a result, surrounding operations may not fuse with the Triton kernel.

CUDA Graphs

CUDA graphs reduce launch overhead by capturing a fixed sequence of GPU work and replaying it. For inference with repeated shapes, this can improve tokens per second.

The code must avoid CPU synchronization during graph capture. Operations such as `.item()` can break graph capture because they move values from GPU to CPU.

A simplified CUDA graph pattern is

```
g = torch.cuda.CUDAGraph()

static_x = torch.empty_like(x)
static_y = torch.empty_like(y)

with torch.cuda.graph(g):
    static_y.copy_(model(static_x))

static_x.copy_(new_x)
g.replay()
```

The lecture reports that manual CUDA graph usage can sometimes outperform relying only on `torch.compile` overhead reduction.

Kernel Selection by Batch Size

Different batch sizes prefer different kernels. A useful mental model is

$$\text{Kernel}(M) = \begin{cases} \text{GMV}, & M \approx 1, \\ \text{Split-}K\text{ GEMM}, & 2 \leq M \leq 32, \\ \text{GEMM}, & M > 32. \end{cases}$$

The exact boundaries depend on GPU architecture, matrix shape, group size, bit width, activation dtype, and packing format. Therefore, the system should benchmark candidate kernels and cache the best choice.

Autotune Caching

Triton autotuning can be expensive. The result should be cached using a key that includes relevant shape and hardware information:

- GPU architecture,

- M, N, and K,
- activation dtype,
- weight bit width,
- group size,
- kernel family,
- split-K factor,
- block sizes.

The lecture notes that Triton config objects may not be directly serializable. A practical implementation can store simplified string or dictionary representations.

Common Misconceptions

Quantization Alone Makes Inference Fast

Quantization reduces storage, but runtime speed requires kernels that exploit the compact representation. If the implementation dequantizes into FP16 first, much of the memory benefit is lost.

Triton Automatically Produces Optimal Kernels

Triton is easier to write than CUDA, but it still requires performance engineering. Load order, memory layout, cache hints, and autotuning can make large differences.

One Kernel Is Enough

Prefill, decode, and batched decode have different bottlenecks. A single kernel is unlikely to be optimal for all regimes.

Bit Packing Is Only a Storage Detail

Packing determines how the kernel loads and reconstructs data. A poor packing layout can dominate performance.

Lower Bit Width Is Always Faster

Lower bit width reduces memory traffic, but it can increase unpacking complexity. Non-power-of-two bit widths such as 3-bit and 5-bit require extra handling.

Autotune Always Finds the Best End-to-End Result

Autotuning may pick a kernel that is fast in isolation but not best after `torch.compile`, CUDA graphs, or full model integration. Manual validation remains necessary.

Practical Takeaways

1. Fuse unpacking, dequantization, and matrix multiplication.
2. Avoid materializing the full dequantized weight matrix.
3. Use GEMM-style kernels for prefill.
4. Use GMV-style kernels for single-token decode.
5. Use split-K GEMM for small-batch decode.
6. Make transposed weights contiguous before benchmarking.
7. Choose one packing format that works across all required kernels.
8. Treat load order as a possible autotuning parameter.
9. Use cache hints such as `evict_last` for small reused activations.
10. Carefully prune invalid autotune configurations.
11. Zero-initialize outputs before atomic accumulation.
12. Cache kernel-selection and autotune results.
13. Test across GPU architectures because behavior can differ across RTX, A100, and H100-class GPUs.

Project or Experiment Ideas

Experiment 1: Naive Versus Fused Dequantization

Implement two quantized linear layers:

1. Dequantize the full weight matrix and call PyTorch matmul.
2. Fuse unpacking, dequantization, and matmul in Triton.

Benchmark both for $M = 1$, $M = 8$, $M = 32$, and $M = 512$.

Experiment 2: Load Order Autotuning

Benchmark two kernel variants:

```
# Variant A
a = tl.load(a_ptrs)
```

```
b = tl.load(b_ptrs)

# Variant B
b = tl.load(b_ptrs)
a = tl.load(a_ptrs)
```

Measure whether the faster order changes across GPUs.

Experiment 3: Eviction Policy

Compare a GMV kernel with and without

```
eviction_policy="evict_last"
```

on activation loads.

Experiment 4: Group Size Sweep

Benchmark group sizes

$$g \in \{32, 64, 128, K\}.$$

Measure both numerical error and runtime. Smaller groups should usually improve accuracy but increase scale and zero-point traffic.

Experiment 5: Split-K Sweep

Sweep split-K factors

$$P \in \{1, 2, 4, 8\}.$$

Benchmark small batch sizes such as

$$M \in \{1, 2, 4, 8, 16, 32\}.$$

Find where split-K beats ordinary GEMM and GMV.

Experiment 6: Three-Bit Reconstruction

Implement 3-bit weights using a 2-bit packed tensor plus a 1-bit packed tensor. Reconstruct inside the Triton kernel and compare speed against 4-bit weights.

Final Mental Model

Low-bit Triton kernels are about reducing memory traffic without losing the savings to inefficient unpacking, bad layouts, or framework overhead. The mathematical operation is simple:

$$Y = X\widehat{W}, \qquad \widehat{W} = S \odot (W_q - Z).$$

The engineering challenge is that W_q is packed, scales and zero points may be grouped, batch size changes the bottleneck, and different GPUs reward different memory behavior.

For prefill, think in terms of tiled GEMM and tensor cores. For single-token decode, think in terms of streaming packed weights as fast as possible while reusing a tiny activation vector. For small-batch decode, think in terms of split-K: create more parallelism over the reduction dimension and combine partial results with atomics.

> Triton is productive, but it is not magic. Fast low-bit kernels require understanding quantization math, bit packing, memory layout, cache behavior, autotuning, atomic accumulation, and the GPU execution model.